Unicorn-Engine API Documentation

Version	1.0.3

Official API document by kabeor

PDF File

Unicorn Engine是一个轻量级, 多平台, 多架构的CPU模拟器框架，当前版本基于Qemu 2.0.x开发，它可以代替CPU模拟代码的执行，常用于恶意代码分析，Fuzzing等，该项目被用于Qiling虚拟框架，Radare2逆向分析框架，GEF(gdb的pwn分析插件)，Pwndbg，Angr符号执行框架等多个著名项目。

0x0 开发准备

Unicorn官网: http://www.unicorn-engine.org

自行编译lib和dll

源码：https://github.com/unicorn-engine/unicorn/archive/master.zip

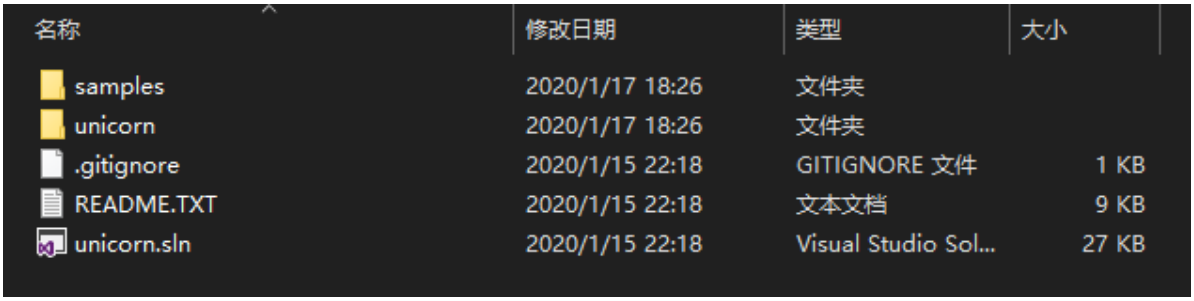
下载后解压

文件结构如下：

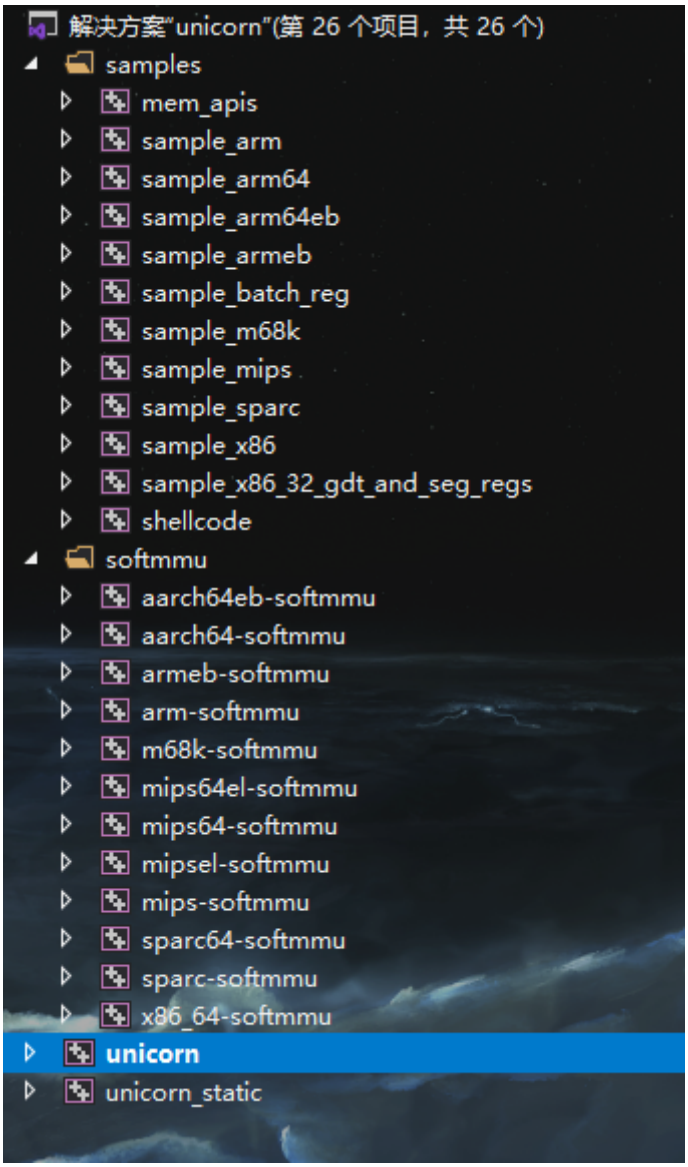
```
. <- 主要引擎core engine + README + 编译文档COMPILE.TXT 等
├── bindings <- 绑定
│ ├── dotnet <- .Net 绑定 + 测试代码
│ ├── go <- go 绑定 + 测试代码
│ ├── haskell <- Haskell 绑定 + 测试代码
│ ├── java <- Java 绑定 + 测试代码
│ ├── pascal <- Pascal 绑定 + 测试代码
│ ├── python <- Python 绑定 + 测试代码
│ ├── ruby <- Ruby 绑定 + 测试代码
│ ├── rust <- Rust 绑定 + 测试代码
│ └── vb6 <- VB6 绑定 + 测试代码
├── docs <- 文档，主要是Unicorn的实现思路
├── include <- C头文件
├── msvc <- Microsoft Visual Studio 支持（Windows）
├── out <- Build 输出
├── qemu <- qemu(已修改)源码
├── samples <- Unicorn使用示例
└── tests <- C语言测试用例
```

下面演示Windows10使用Visual Studio 2019编译

打开msvc文件夹，内部结构如下

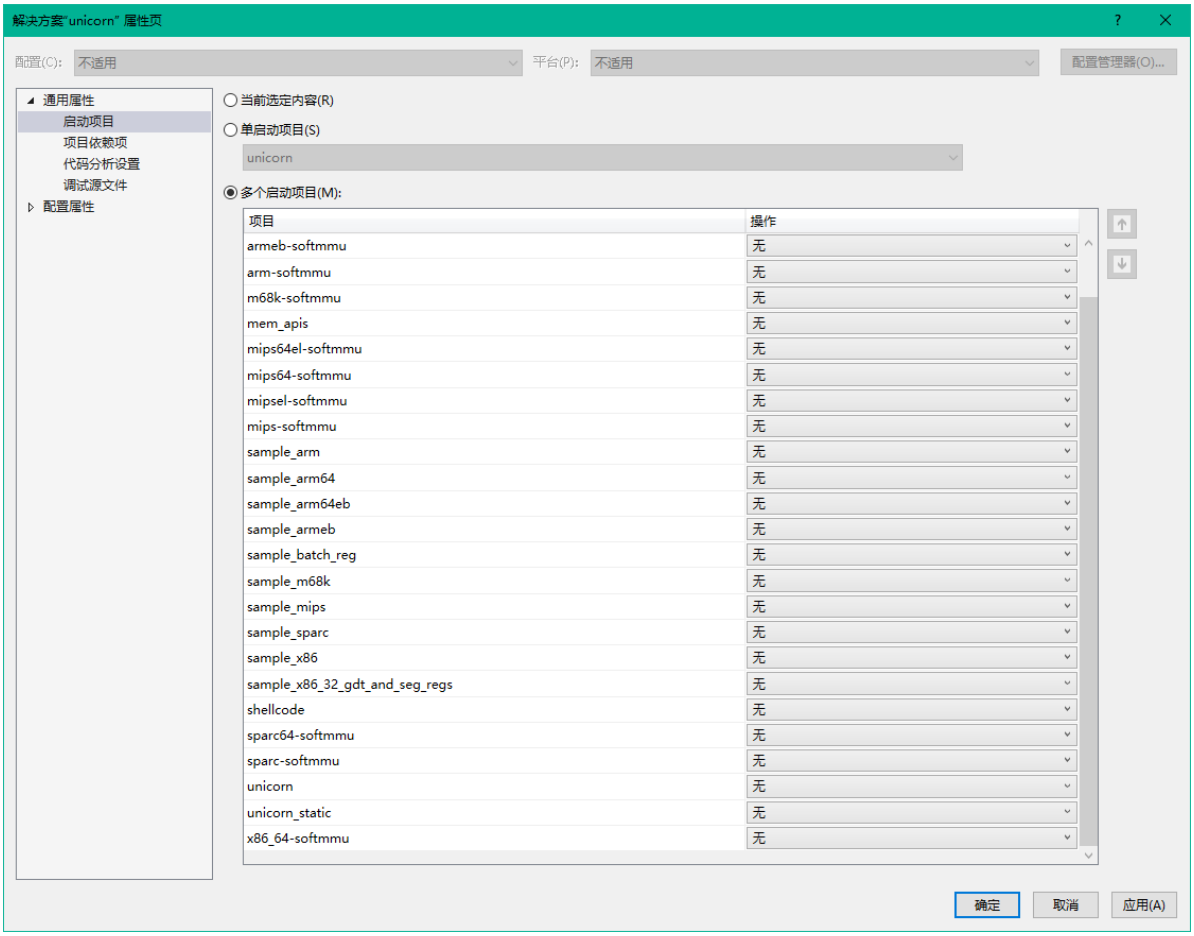


VS打开unicorn.sln项目文件，解决方案自动载入这些



如果都需要的话，直接编译就好了，只需要其中几种，则右键解决方案->属性->配置属性->生成选项 中勾选你需要的支持项即可

也可在启动项目中配置多个项目操作 如下



编译后会在当前文件夹Debug目录下生成unicorn.lib静态编译库和unicorn.dll动态库这样就可以开始使用Unicorn进行开发了

官方目前提供的最新已编译版本为1.0.3版本，可自己编辑最新版本源码，以获得更多可用API。

> Win32：https://github.com/unicorn-engine/unicorn/releases/download/1.0.1/unicorn-1.0.3-win32.zip

> Win64：https://github.com/unicorn-engine/unicorn/releases/download/1.0.1/unicorn-1.0.3-win64.zip

注意： 选x32或x64将影响后面开发的架构

点击编译，到unicorn\msvc\x32或x64\Debug或Release下找unicorn.dll和unicorn.lib即可

引擎调用测试

新建一个VS项目，将..\unicorn-master\include\unicorn中的头文件以及编译好的lib和dll文件全部拷贝到新建项目的主目录下



名称	修改日期	类型	大小
.vs	2020/1/17 17:23	文件夹	
Debug	2020/1/17 17:30	文件夹	
unicorn	2020/1/17 17:25	文件夹	
x64	2020/1/17 17:30	文件夹	
unicorn.dll	2020/1/17 17:02	应用程序扩展	4,479 KB
unicorn.lib	2020/1/17 17:02	Object File Library	7 KB
Unicorn_Demo.cpp	2020/1/17 17:38	c_file	3 KB
Unicorn_Demo.sln	2020/1/17 17:23	Visual Studio Sol...	2 KB
Unicorn_Demo.vcxproj	2020/1/17 17:32	VC++ Project	8 KB
Unicorn_Demo.vcxproj.filters	2020/1/17 17:30	VC++ Project Fil...	2 KB
Unicorn_Demo.vcxproj.user	2020/1/17 17:23	Per-User Project...	1 KB

在VS解决方案中，头文件添加现有项unicorn.h，资源文件中添加unicorn.lib，重新生成解决方案



接下来测试我们生成的Unicorn引擎

主文件代码如下

▶ Code

```
#include <iostream>
#include "unicorn/unicorn.h"

// 要模拟的指令
#define X86_CODE32 "\x41\x4a" // INC ecx; DEC edx

// 起始地址
#define ADDRESS 0x1000000

int main()
{
    uc_engine* uc;
    uc_err err;
    int r_ecx = 0x1234;     // ECX 寄存器
    int r_edx = 0x7890;     // EDX 寄存器

    printf("Emulate i386 code\n");

    // X86-32bit 模式初始化模拟
    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
            return -1;
```

```
    }

    // 给模拟器申请 2MB 内存
    uc_mem_map(uc, ADDRESS, 2 * 1024 * 1024, UC_PROT_ALL);

    // 将要模拟的指令写入内存
    if (uc_mem_write(uc, ADDRESS, X86_CODE32, sizeof(X86_CODE32) - 1)) {
        printf("Failed to write emulation code to memory, quit!\n");
        return -1;
    }

    // 初始化寄存器
    uc_reg_write(uc, UC_X86_REG_ECX, &r_ecx);
    uc_reg_write(uc, UC_X86_REG_EDX, &r_edx);

    printf(">>> ECX = 0x%x\n", r_ecx);
    printf(">>> EDX = 0x%x\n", r_edx);

     // 模拟代码
    err = uc_emu_start(uc, ADDRESS, ADDRESS + sizeof(X86_CODE32) - 1, 0, 0);
    if (err) {
        printf("Failed on uc_emu_start() with error returned %u: %s\n",
        err, uc_strerror(err));
    }

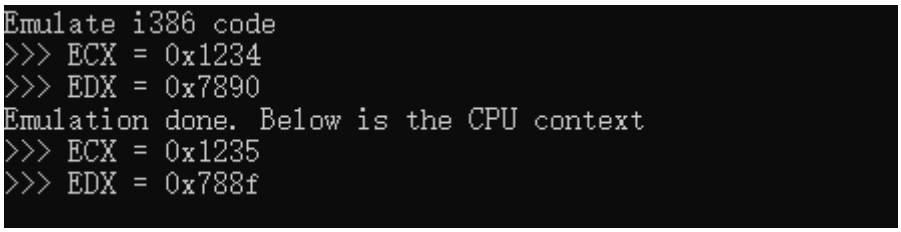
    // 打印寄存器值
    printf("Emulation done. Below is the CPU context\n");

    uc_reg_read(uc, UC_X86_REG_ECX, &r_ecx);
    uc_reg_read(uc, UC_X86_REG_EDX, &r_edx);
    printf(">>> ECX = 0x%x\n", r_ecx);
    printf(">>> EDX = 0x%x\n", r_edx);

    uc_close(uc);

    return 0;
}
```

运行结果如下



ecx+1和edx-1成功模拟。

0x1 数据类型

> 索引

uc_arch

uc_mode

uc_err

uc_mem_type

uc_hook_type

Hook Types

uc_mem_region

uc_query_type

uc_context

uc_prot

---

uc_arch

架构选择

▶ Code

```
typedef enum uc_arch {
    UC_ARCH_ARM = 1,    // ARM 架构 (包括 Thumb, Thumb-2)
    UC_ARCH_ARM64,      // ARM-64, 也称 AArch64
    UC_ARCH_MIPS,       // Mips 架构
    UC_ARCH_X86,        // X86 架构 (包括 x86 & x86-64)
    UC_ARCH_PPC,        // PowerPC 架构 (暂不支持)
    UC_ARCH_SPARC,      // Sparc 架构
    UC_ARCH_M68K,       // M68K 架构
    UC_ARCH_MAX,
} uc_arch;
```

uc_mode

模式选择

▶ Code

```
typedef enum uc_mode {
    UC_MODE_LITTLE_ENDIAN = 0,    // 小端序模式 (默认)
    UC_MODE_BIG_ENDIAN = 1 << 30, // 大端序模式

    // arm / arm64
    UC_MODE_ARM = 0,              // ARM 模式
    UC_MODE_THUMB = 1 << 4,       // THUMB 模式 (包括 Thumb-2)
    UC_MODE_MCLASS = 1 << 5,      // ARM's Cortex-M 系列 (暂不支持)
    UC_MODE_V8 = 1 << 6,          // ARMv8 A32 encodings for ARM (暂不支持)

    // arm (32bit) cpu 类型
    UC_MODE_ARM926 = 1 << 7,      // ARM926 CPU 类型
    UC_MODE_ARM946 = 1 << 8,      // ARM946 CPU 类型
    UC_MODE_ARM1176 = 1 << 9,     // ARM1176 CPU 类型

    // mips
    UC_MODE_MICRO = 1 << 4,       // MicroMips 模式 (暂不支持)
    UC_MODE_MIPS3 = 1 << 5,       // Mips III ISA (暂不支持)
    UC_MODE_MIPS32R6 = 1 << 6,    // Mips32r6 ISA (暂不支持)
    UC_MODE_MIPS32 = 1 << 2,      // Mips32 ISA
    UC_MODE_MIPS64 = 1 << 3,      // Mips64 ISA
```

```
    // x86 / x64
    UC_MODE_16 = 1 << 1,          // 16-bit 模式
    UC_MODE_32 = 1 << 2,          // 32-bit 模式
    UC_MODE_64 = 1 << 3,          // 64-bit 模式

    // ppc
    UC_MODE_PPC32 = 1 << 2,       // 32-bit 模式 (暂不支持)
    UC_MODE_PPC64 = 1 << 3,       // 64-bit 模式 (暂不支持)
    UC_MODE_QPX = 1 << 4,         // Quad Processing eXtensions 模式 (暂不支持)

    // sparc
    UC_MODE_SPARC32 = 1 << 2,     // 32-bit 模式
    UC_MODE_SPARC64 = 1 << 3,     // 64-bit 模式
    UC_MODE_V9 = 1 << 4,          // SparcV9 模式 (暂不支持)

    // m68k
} uc_mode;
```

uc_err

错误类型，是uc_errno()的返回值

▶ Code

```
typedef enum uc_err {
    UC_ERR_OK = 0,   // 无错误
    UC_ERR_NOMEM,      // 内存不足: uc_open(), uc_emulate()
    UC_ERR_ARCH,     // 不支持的架构: uc_open()
    UC_ERR_HANDLE,   // 不可用句柄
    UC_ERR_MODE,     // 不可用/不支持架构: uc_open()
    UC_ERR_VERSION,  // 不支持版本 (中间件)
    UC_ERR_READ_UNMAPPED, // 由于在未映射的内存上读取而退出模拟: uc_emu_start()
    UC_ERR_WRITE_UNMAPPED, // 由于在未映射的内存上写入而退出模拟: uc_emu_start()
    UC_ERR_FETCH_UNMAPPED, // 由于在未映射的内存中获取数据而退出模拟: uc_emu_start()
    UC_ERR_HOOK,    // 无效的hook类型: uc_hook_add()
    UC_ERR_INSN_INVALID, // 由于指令无效而退出模拟: uc_emu_start()
    UC_ERR_MAP, // 无效的内存映射: uc_mem_map()
    UC_ERR_WRITE_PROT, // 由于UC_MEM_WRITE_PROT冲突而停止模拟: uc_emu_start()
    UC_ERR_READ_PROT, // 由于UC_MEM_READ_PROT冲突而停止模拟: uc_emu_start()
    UC_ERR_FETCH_PROT, // 由于UC_MEM_FETCH_PROT冲突而停止模拟: uc_emu_start()
    UC_ERR_ARG,     // 提供给uc_xxx函数的无效参数
    UC_ERR_READ_UNALIGNED,  // 未对齐读取
    UC_ERR_WRITE_UNALIGNED,  // 未对齐写入
    UC_ERR_FETCH_UNALIGNED,  // 未对齐的提取
    UC_ERR_HOOK_EXIST,  // 此事件的钩子已经存在
    UC_ERR_RESOURCE,    // 资源不足: uc_emu_start()
    UC_ERR_EXCEPTION, // 未处理的CPU异常
    UC_ERR_TIMEOUT // 模拟超时
} uc_err;
```

uc_mem_type

UC_HOOK_MEM_*的所有内存访问类型

▶ Code

```
typedef enum uc_mem_type {
    UC_MEM_READ = 16,   // 内存从..读取
    UC_MEM_WRITE,       // 内存写入到..
    UC_MEM_FETCH,       // 内存被获取
    UC_MEM_READ_UNMAPPED,    // 未映射内存从..读取
    UC_MEM_WRITE_UNMAPPED,   // 未映射内存写入到..
    UC_MEM_FETCH_UNMAPPED,   // 未映射内存被获取
    UC_MEM_WRITE_PROT,  // 内存写保护，但是已映射
    UC_MEM_READ_PROT,   // 内存读保护，但是已映射
    UC_MEM_FETCH_PROT,  // 内存不可执行，但是已映射
    UC_MEM_READ_AFTER,   // 内存从（成功访问的地址）读入
} uc_mem_type;
```

uc_hook_type

uc_hook_add()的所有hook类型参数

▶ Code

```
typedef enum uc_hook_type {
    // Hook 所有中断/syscall 事件
    UC_HOOK_INTR = 1 << 0,
    // Hook 一条特定的指令 - 只支持非常小的指令子集
    UC_HOOK_INSN = 1 << 1,
    // Hook 一段代码
    UC_HOOK_CODE = 1 << 2,
    // Hook 基本块
    UC_HOOK_BLOCK = 1 << 3,
    // 用于在未映射的内存上读取内存的Hook
    UC_HOOK_MEM_READ_UNMAPPED = 1 << 4,
    // Hook 无效的内存写事件
    UC_HOOK_MEM_WRITE_UNMAPPED = 1 << 5,
    // Hook 执行事件的无效内存
    UC_HOOK_MEM_FETCH_UNMAPPED = 1 << 6,
    // Hook 读保护的内存
    UC_HOOK_MEM_READ_PROT = 1 << 7,
    // Hook 写保护的内存
    UC_HOOK_MEM_WRITE_PROT = 1 << 8,
    // Hook 不可执行内存上的内存
    UC_HOOK_MEM_FETCH_PROT = 1 << 9,
    // Hook 内存读取事件
    UC_HOOK_MEM_READ = 1 << 10,
    // Hook 内存写入事件
    UC_HOOK_MEM_WRITE = 1 << 11,
    // Hook 内存获取执行事件
    UC_HOOK_MEM_FETCH = 1 << 12,
    // Hook 内存读取事件，只允许能成功访问的地址
    // 成功读取后将触发回调
    UC_HOOK_MEM_READ_AFTER = 1 << 13,
    // Hook 无效指令异常
    UC_HOOK_INSN_INVALID = 1 << 14,
```

```
} uc_hook_type;
```

hook_types

宏定义Hook类型

▶ Code

```
// Hook 所有未映射内存访问的事件
#define UC_HOOK_MEM_UNMAPPED (UC_HOOK_MEM_READ_UNMAPPED +
UC_HOOK_MEM_WRITE_UNMAPPED + UC_HOOK_MEM_FETCH_UNMAPPED)
// Hook 所有对受保护内存的非法访问事件
#define UC_HOOK_MEM_PROT (UC_HOOK_MEM_READ_PROT + UC_HOOK_MEM_WRITE_PROT +
UC_HOOK_MEM_FETCH_PROT)
// Hook 所有非法读取存储器的事件
#define UC_HOOK_MEM_READ_INVALID (UC_HOOK_MEM_READ_PROT +
UC_HOOK_MEM_READ_UNMAPPED)
// Hook 所有非法写入存储器的事件
#define UC_HOOK_MEM_WRITE_INVALID (UC_HOOK_MEM_WRITE_PROT +
UC_HOOK_MEM_WRITE_UNMAPPED)
// Hook 所有非法获取内存的事件
#define UC_HOOK_MEM_FETCH_INVALID (UC_HOOK_MEM_FETCH_PROT +
UC_HOOK_MEM_FETCH_UNMAPPED)
// Hook 所有非法的内存访问事件
#define UC_HOOK_MEM_INVALID (UC_HOOK_MEM_UNMAPPED + UC_HOOK_MEM_PROT)
// Hook 所有有效内存访问的事件
// 注意: UC_HOOK_MEM_READ 在 UC_HOOK_MEM_READ_PROT 和 UC_HOOK_MEM_READ_UNMAPPED 之
前触发 ,
//       因此这个Hook可能会触发一些无效的读取。
#define UC_HOOK_MEM_VALID (UC_HOOK_MEM_READ + UC_HOOK_MEM_WRITE +
UC_HOOK_MEM_FETCH)
```

uc_mem_region

由uc_mem_map()和uc_mem_map_ptr()映射内存区域
使用uc_mem_regions()检索该内存区域的列表

▶ Code

```
typedef struct uc_mem_region {
    uint64_t begin; // 区域起始地址 (包括)
    uint64_t end;   // 区域结束地址 (包括)
    uint32_t perms; // 区域的内存权限
} uc_mem_region;
```

uc_query_type

uc_query()的所有查询类型参数

▶ Code

```
typedef enum uc_query_type {
    // 动态查询当前硬件模式
    UC_QUERY_MODE = 1,
    UC_QUERY_PAGE_SIZE,
    UC_QUERY_ARCH,
} uc_query_type;
```

uc_context

与uc_context_*()一起使用，管理CPU上下文的不透明存储

▶ Code

```
struct uc_context;
typedef struct uc_context uc_context;
```

uc_prot

新映射区域的权限

▶ Code

```
typedef enum uc_prot {
   UC_PROT_NONE = 0,    //无
   UC_PROT_READ = 1,    //读取
   UC_PROT_WRITE = 2,   //写入
   UC_PROT_EXEC = 4,    //可执行
   UC_PROT_ALL = 7,     //所有权限
} uc_prot;
```

0x2 API

> 索引

uc_version

uc_arch_supported

uc_open

uc_close

uc_query

uc_errno

uc_strerror

uc_reg_write

uc_reg_read

uc_reg_write_batch

uc_reg_read_batch

uc_mem_write

uc_mem_read

uc_emu_start

uc_emu_stop

uc_hook_add

uc_hook_del

uc_mem_map

uc_mem_map_ptr

uc_mem_unmap

uc_mem_protect

uc_mem_regions

uc_free

uc_context_alloc

uc_context_save

uc_context_restore

uc_context_size

uc_context_free

---

uc_version

```
unsigned int uc_version(unsigned int *major, unsigned int *minor);
```

用于返回Unicorn API主次版本信息

```
@major: API主版本号
@minor: API次版本号
@return 16进制数，计算方式 (major << 8 | minor)

提示: 该返回值可以和宏UC_MAKE_VERSION比较
```

▶ 源码实现

```
unsigned int uc_version(unsigned int *major, unsigned int *minor)
{
    if (major != NULL && minor != NULL) {
        *major = UC_API_MAJOR;  //宏
        *minor = UC_API_MINOR;  //宏
    }

    return (UC_API_MAJOR << 8) + UC_API_MINOR;   //(major << 8 | minor)
}
```

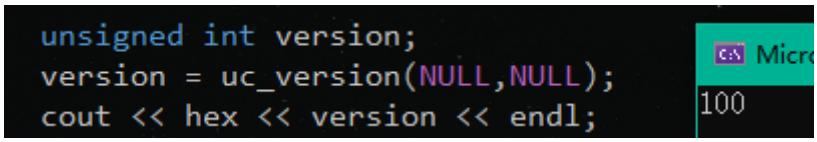
编译后不可更改，不接受自定义版本

使用示例：

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    unsigned int version;
    version = uc_version(NULL,NULL);
    cout << hex << version << endl;
    return 0;
}
```

输出：



得到版本号1.0.0

uc_arch_supported

```
bool uc_arch_supported(uc_arch arch);
```

确定Unicorn是否支持当前架构

```
 @arch: 架构类型 (UC_ARCH_*)
 @return 如果支持返回True
```

▶ 源码实现

```
bool uc_arch_supported(uc_arch arch)
{
    switch (arch) {
#ifdef UNICORN_HAS_ARM
        case UC_ARCH_ARM:   return true;
#endif
#ifdef UNICORN_HAS_ARM64
        case UC_ARCH_ARM64: return true;
#endif
#ifdef UNICORN_HAS_M68K
        case UC_ARCH_M68K:  return true;
#endif
#ifdef UNICORN_HAS_MIPS
        case UC_ARCH_MIPS:  return true;
#endif
#ifdef UNICORN_HAS_PPC
        case UC_ARCH_PPC:   return true;
#endif
#ifdef UNICORN_HAS_SPARC
        case UC_ARCH_SPARC: return true;
#endif
#ifdef UNICORN_HAS_X86
```

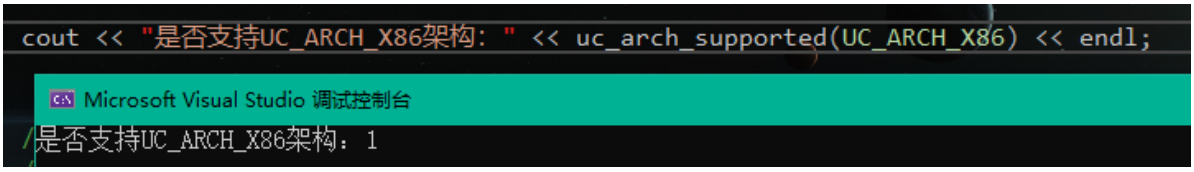
```
        case UC_ARCH_X86:    return true;
#endif
        /* 无效或禁用架构 */
        default:             return false;
    }
}
```

使用示例：

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    cout << "是否支持UC_ARCH_X86架构：" << uc_arch_supported(UC_ARCH_X86) << endl;
    return 0;
}
```

输出：



uc_open

```
uc_err uc_open(uc_arch arch, uc_mode mode, uc_engine **uc);
```

创建新的Unicorn实例

```
@arch: 架构类型 (UC_ARCH_*)
@mode: 硬件模式. 由 UC_MODE_* 组合
@uc: 指向 uc_engine 的指针, 返回时更新

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_open(uc_arch arch, uc_mode mode, uc_engine **result)
{
    struct uc_struct *uc;

    if (arch < UC_ARCH_MAX) {
        uc = calloc(1, sizeof(*uc));  //申请内存
        if (!uc) {
            // 内存不足
            return UC_ERR_NOMEM;
        }

        uc->errnum = UC_ERR_OK;
        uc->arch = arch;
```

```
        uc->mode = mode;

        // 初始化
        // uc->ram_list = { .blocks = QTAILQ_HEAD_INITIALIZER(ram_list.blocks)
};
        uc->ram_list.blocks.tqh_first = NULL;
        uc->ram_list.blocks.tqh_last = &(uc->ram_list.blocks.tqh_first);

        uc->memory_listeners.tqh_first = NULL;
        uc->memory_listeners.tqh_last = &uc->memory_listeners.tqh_first;

        uc->address_spaces.tqh_first = NULL;
        uc->address_spaces.tqh_last = &uc->address_spaces.tqh_first;

        switch(arch) {   // 根据架构进行预处理
            default:
                break;
#ifdef UNICORN_HAS_M68K
            case UC_ARCH_M68K:
                if ((mode & ~UC_MODE_M68K_MASK) ||
                        !(mode & UC_MODE_BIG_ENDIAN)) {
                    free(uc);
                    return UC_ERR_MODE;
                }
                uc->init_arch = m68k_uc_init;
                break;
#endif
#ifdef UNICORN_HAS_X86
            case UC_ARCH_X86:
                if ((mode & ~UC_MODE_X86_MASK) ||
                        (mode & UC_MODE_BIG_ENDIAN) ||
                        !(mode & (UC_MODE_16|UC_MODE_32|UC_MODE_64))) {
                    free(uc);
                    return UC_ERR_MODE;
                }
                uc->init_arch = x86_uc_init;
                break;
#endif
#ifdef UNICORN_HAS_ARM
            case UC_ARCH_ARM:
                if ((mode & ~UC_MODE_ARM_MASK)) {
                    free(uc);
                    return UC_ERR_MODE;
                }
                if (mode & UC_MODE_BIG_ENDIAN) {
                    uc->init_arch = armeb_uc_init;
                } else {
                    uc->init_arch = arm_uc_init;
                }

                if (mode & UC_MODE_THUMB)
                    uc->thumb = 1;
                break;
#endif
#ifdef UNICORN_HAS_ARM64
            case UC_ARCH_ARM64:
                if (mode & ~UC_MODE_ARM_MASK) {
                    free(uc);
```

```
                    return UC_ERR_MODE;
                }
                if (mode & UC_MODE_BIG_ENDIAN) {
                    uc->init_arch = arm64eb_uc_init;
                } else {
                    uc->init_arch = arm64_uc_init;
                }
                break;
#endif

#if defined(UNICORN_HAS_MIPS) || defined(UNICORN_HAS_MIPSEL) ||
defined(UNICORN_HAS_MIPS64) || defined(UNICORN_HAS_MIPS64EL)
            case UC_ARCH_MIPS:
                if ((mode & ~UC_MODE_MIPS_MASK) ||
                        !(mode & (UC_MODE_MIPS32|UC_MODE_MIPS64))) {
                    free(uc);
                    return UC_ERR_MODE;
                }
                if (mode & UC_MODE_BIG_ENDIAN) {
#ifdef UNICORN_HAS_MIPS
                    if (mode & UC_MODE_MIPS32)
                        uc->init_arch = mips_uc_init;
#endif
#ifdef UNICORN_HAS_MIPS64
                    if (mode & UC_MODE_MIPS64)
                        uc->init_arch = mips64_uc_init;
#endif
                } else {    // 小端序
#ifdef UNICORN_HAS_MIPSEL
                    if (mode & UC_MODE_MIPS32)
                        uc->init_arch = mipsel_uc_init;
#endif
#ifdef UNICORN_HAS_MIPS64EL
                    if (mode & UC_MODE_MIPS64)
                        uc->init_arch = mips64el_uc_init;
#endif
                }
                break;
#endif

#ifdef UNICORN_HAS_SPARC
            case UC_ARCH_SPARC:
                if ((mode & ~UC_MODE_SPARC_MASK) ||
                        !(mode & UC_MODE_BIG_ENDIAN) ||
                        !(mode & (UC_MODE_SPARC32|UC_MODE_SPARC64))) {
                    free(uc);
                    return UC_ERR_MODE;
                }
                if (mode & UC_MODE_SPARC64)
                    uc->init_arch = sparc64_uc_init;
                else
                    uc->init_arch = sparc_uc_init;
                break;
#endif
        }

        if (uc->init_arch == NULL) {
            return UC_ERR_ARCH;
```

```
        }

        if (machine_initialize(uc))
            return UC_ERR_RESOURCE;

        *result = uc;

        if (uc->reg_reset)
            uc->reg_reset(uc);

        return UC_ERR_OK;
    } else {
        return UC_ERR_ARCH;
    }
}
```

注意： uc_open会申请堆内存，使用完必须用uc_close释放，否则会发生泄露

使用示例：

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    uc_engine* uc;
    uc_err err;

    //// 初始化 X86-32bit 模式模拟器
    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
            return -1;
    }

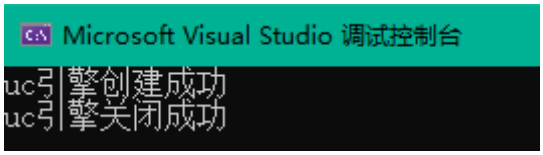
    if (!err)
        cout << "uc引擎创建成功" << endl;

    //// 关闭uc
    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }

    if (!err)
        cout << "uc引擎关闭成功" << endl;

    return 0;
}
```

输出



uc_close

```
uc_err uc_close(uc_engine *uc);
```

关闭一个uc实例，将释放内存。关闭后无法恢复。

```
@uc: 指向由 uc_open() 返回的指针

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_close(uc_engine *uc)
{
    int i;
    struct list_item *cur;
    struct hook *hook;

    // 清理内部数据
    if (uc->release)
        uc->release(uc->tcg_ctx);
    g_free(uc->tcg_ctx);

    // 清理 CPU.
    g_free(uc->cpu->tcg_as_listener);
    g_free(uc->cpu->thread);

    // 清理所有 objects.
    OBJECT(uc->machine_state->accelerator)->ref = 1;
    OBJECT(uc->machine_state)->ref = 1;
    OBJECT(uc->owner)->ref = 1;
    OBJECT(uc->root)->ref = 1;

    object_unref(uc, OBJECT(uc->machine_state->accelerator));
    object_unref(uc, OBJECT(uc->machine_state));
    object_unref(uc, OBJECT(uc->cpu));
    object_unref(uc, OBJECT(&uc->io_mem_notdirty));
    object_unref(uc, OBJECT(&uc->io_mem_unassigned));
    object_unref(uc, OBJECT(&uc->io_mem_rom));
    object_unref(uc, OBJECT(uc->root));

    // 释放内存
    g_free(uc->system_memory);

    // 释放相关线程
    if (uc->qemu_thread_data)
        g_free(uc->qemu_thread_data);

    // 释放其他数据
```

```
    free(uc->l1_map);

    if (uc->bounce.buffer) {
        free(uc->bounce.buffer);
    }

    g_hash_table_foreach(uc->type_table, free_table, uc);
    g_hash_table_destroy(uc->type_table);

    for (i = 0; i < DIRTY_MEMORY_NUM; i++) {
        free(uc->ram_list.dirty_memory[i]);
    }

    // 释放hook和hook列表
    for (i = 0; i < UC_HOOK_MAX; i++) {
        cur = uc->hook[i].head;
        // hook 可存在于多个列表，可通过计数获取释放的时间
        while (cur) {
            hook = (struct hook *)cur->data;
            if (--hook->refs == 0) {
                free(hook);
            }
            cur = cur->next;
        }
        list_clear(&uc->hook[i]);
    }

    free(uc->mapped_blocks);

    // 最后释放uc自身
    memset(uc, 0, sizeof(*uc));
    free(uc);

    return UC_ERR_OK;
}
```

使用实例同uc_open()

uc_query

```
uc_err uc_query(uc_engine *uc, uc_query_type type, size_t *result);
```

查询引擎的内部状态

```
 @uc: uc_open() 返回的句柄
 @type: uc_query_type 中枚举的类型

 @result: 保存被查询的内部状态的指针

 @return: 成功则返回UC_ERR_OK ， 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_query(uc_engine *uc, uc_query_type type, size_t *result)
```

```
{
    if (type == UC_QUERY_PAGE_SIZE) {
        *result = uc->target_page_size;
        return UC_ERR_OK;
    }

    if (type == UC_QUERY_ARCH) {
        *result = uc->arch;
        return UC_ERR_OK;
    }

    switch(uc->arch) {
#ifdef UNICORN_HAS_ARM
        case UC_ARCH_ARM:
            return uc->query(uc, type, result);
#endif
        default:
            return UC_ERR_ARG;
    }

    return UC_ERR_OK;
}
```

使用示例:

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;
int main()
{
    uc_engine* uc;
    uc_err err;

    //// Initialize emulator in X86-32bit mode
    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例创建成功" << endl;

    size_t result[] = {0};
    err = uc_query(uc, UC_QUERY_ARCH, result);   // 查询架构
    if (!err)
        cout << "查询成功: " << *result << endl;

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例关闭成功" << endl;

    return 0;
}
```

输出



架构查询结果为4，对应的正是UC_ARCH_X86

uc_errno

```
uc_err uc_errno(uc_engine *uc);
```

当某个API函数失败时，报告最后的错误号，一旦被访问，uc_errno可能不会保留原来的值。

```
@uc: uc_open() 返回的句柄

@return: 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_errno(uc_engine *uc)
{
    return uc->errnum;
}
```

使用示例:

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    uc_engine* uc;
    uc_err err;

    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例创建成功" << endl;

    err = uc_errno(uc);
    cout << "错误号:  " << err << endl;

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }
```

```
    if (!err)
        cout << "uc实例关闭成功" << endl;

    return 0;
}
```

输出



无错误，输出错误号为0

uc_strerror

```
const char *uc_strerror(uc_err code);
```

返回给定错误号的解释

```
 @code: 错误号

 @return: 指向给定错误号的解释的字符串指针
```

▶ 源码实现

```
const char *uc_strerror(uc_err code)
{
    switch(code) {
        default:
            return "Unknown error code";
        case UC_ERR_OK:
            return "OK (UC_ERR_OK)";
        case UC_ERR_NOMEM:
            return "No memory available or memory not present (UC_ERR_NOMEM)";
        case UC_ERR_ARCH:
            return "Invalid/unsupported architecture (UC_ERR_ARCH)";
        case UC_ERR_HANDLE:
            return "Invalid handle (UC_ERR_HANDLE)";
        case UC_ERR_MODE:
            return "Invalid mode (UC_ERR_MODE)";
        case UC_ERR_VERSION:
            return "Different API version between core & binding
(UC_ERR_VERSION)";
        case UC_ERR_READ_UNMAPPED:
            return "Invalid memory read (UC_ERR_READ_UNMAPPED)";
        case UC_ERR_WRITE_UNMAPPED:
            return "Invalid memory write (UC_ERR_WRITE_UNMAPPED)";
        case UC_ERR_FETCH_UNMAPPED:
            return "Invalid memory fetch (UC_ERR_FETCH_UNMAPPED)";
        case UC_ERR_HOOK:
            return "Invalid hook type (UC_ERR_HOOK)";
```

```
        case UC_ERR_INSN_INVALID:
            return "Invalid instruction (UC_ERR_INSN_INVALID)";
        case UC_ERR_MAP:
            return "Invalid memory mapping (UC_ERR_MAP)";
        case UC_ERR_WRITE_PROT:
            return "Write to write-protected memory (UC_ERR_WRITE_PROT)";
        case UC_ERR_READ_PROT:
            return "Read from non-readable memory (UC_ERR_READ_PROT)";
        case UC_ERR_FETCH_PROT:
            return "Fetch from non-executable memory (UC_ERR_FETCH_PROT)";
        case UC_ERR_ARG:
            return "Invalid argument (UC_ERR_ARG)";
        case UC_ERR_READ_UNALIGNED:
            return "Read from unaligned memory (UC_ERR_READ_UNALIGNED)";
        case UC_ERR_WRITE_UNALIGNED:
            return "Write to unaligned memory (UC_ERR_WRITE_UNALIGNED)";
        case UC_ERR_FETCH_UNALIGNED:
            return "Fetch from unaligned memory (UC_ERR_FETCH_UNALIGNED)";
        case UC_ERR_RESOURCE:
            return "Insufficient resource (UC_ERR_RESOURCE)";
        case UC_ERR_EXCEPTION:
            return "Unhandled CPU exception (UC_ERR_EXCEPTION)";
        case UC_ERR_TIMEOUT:
            return "Emulation timed out (UC_ERR_TIMEOUT)";
    }
}
```

使用示例：

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    uc_engine* uc;
    uc_err err;

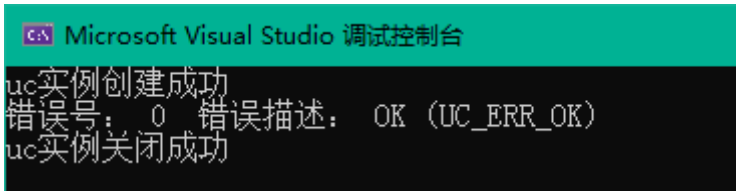
    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例创建成功" << endl;

    err = uc_errno(uc);
    cout << "错误号：  " << err << "  错误描述：  " << uc_strerror(err) <<endl;

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例关闭成功" << endl;
```

```
    return 0;
}
```

输出



uc_reg_write

```
uc_err uc_reg_write(uc_engine *uc, int regid, const void *value);
```

将值写入寄存器

```
@uc: uc_open()返回的句柄
@regid:  将被修改的寄存器ID
@value:  指向寄存器将被修改成的值的指针

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_reg_write(uc_engine *uc, int regid, const void *value)
{
    return uc_reg_write_batch(uc, &regid, (void *const *)&value, 1);
}

uc_err uc_reg_write_batch(uc_engine *uc, int *ids, void *const *vals, int count)
{
    int ret = UC_ERR_OK;
    if (uc->reg_write)
        ret = uc->reg_write(uc, (unsigned int *)ids, vals, count);    //结构体中写
入
    else
        return UC_ERR_EXCEPTION;

    return ret;
}
```

使用示例：

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    uc_engine* uc;
    uc_err err;
```

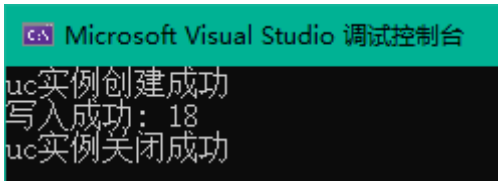
```
    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例创建成功" << endl;

    int r_eax = 0x12;
    err = uc_reg_write(uc, UC_X86_REG_ECX, &r_eax);
    if (!err)
        cout << "写入成功: " << r_eax << endl;

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例关闭成功" << endl;

    return 0;
}
```

输出



uc_reg_read

```
uc_err uc_reg_read(uc_engine *uc, int regid, void *value);
```

读取寄存器的值

```
@uc: uc_open()返回的句柄
@regid:  将被读取的寄存器ID
@value:  指向保存寄存器值的指针

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_reg_read(uc_engine *uc, int regid, void *value)
{
    return uc_reg_read_batch(uc, &regid, &value, 1);
}

uc_err uc_reg_read_batch(uc_engine *uc, int *ids, void **vals, int count)
{
    if (uc->reg_read)
```

```
        uc->reg_read(uc, (unsigned int *)ids, vals, count);
    else
        return -1;

    return UC_ERR_OK;
}
```

使用示例：

```
#include <iostream>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    uc_engine* uc;
    uc_err err;

    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例创建成功" << endl;

    int r_eax = 0x12;
    err = uc_reg_write(uc, UC_X86_REG_ECX, &r_eax);
    if (!err)
        cout << "写入成功: " << r_eax << endl;

    int recv_eax;
    err = uc_reg_read(uc, UC_X86_REG_ECX, &recv_eax);
    if (!err)
        cout << "读取成功: " << recv_eax << endl;

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }
    if (!err)
        cout << "uc实例关闭成功" << endl;

    return 0;
}
```

输出



uc_reg_write_batch

```
uc_err uc_reg_write_batch(uc_engine *uc, int *regs, void *const *vals, int
count);
```

同时将多个值写入多个寄存器

```
@uc: uc_open()返回的句柄
@regid:  存储将被写入的多个寄存器ID的数组
@value:  指向保存多个值的数组的指针
@count: *regs 和 *vals 数组的长度

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_reg_write_batch(uc_engine *uc, int *ids, void *const *vals, int count)
{
    int ret = UC_ERR_OK;
    if (uc->reg_write)
        ret = uc->reg_write(uc, (unsigned int *)ids, vals, count);
    else
        return UC_ERR_EXCEPTION;

    return ret;
}
```

使用示例:

```
#include <iostream>
#include <string>
#include "unicorn/unicorn.h"
using namespace std;

int syscall_abi[] = {
    UC_X86_REG_RAX, UC_X86_REG_RDI, UC_X86_REG_RSI, UC_X86_REG_RDX,
    UC_X86_REG_R10, UC_X86_REG_R8, UC_X86_REG_R9
};

uint64_t vals[7] = { 200, 10, 11, 12, 13, 14, 15 };

void* ptrs[7];

int main()
{
    int i;
    uc_err err;
    uc_engine* uc;

    // set up register pointers
    for (i = 0; i < 7; i++) {
        ptrs[i] = &vals[i];
    }

    if ((err = uc_open(UC_ARCH_X86, UC_MODE_64, &uc))) {
```

```
        uc_perror("uc_open", err);
        return 1;
    }

    // reg_write_batch
    printf("reg_write_batch({200, 10, 11, 12, 13, 14, 15})\n");
    if ((err = uc_reg_write_batch(uc, syscall_abi, ptrs, 7))) {
        uc_perror("uc_reg_write_batch", err);
        return 1;
    }

    // reg_read_batch
    memset(vals, 0, sizeof(vals));
    if ((err = uc_reg_read_batch(uc, syscall_abi, ptrs, 7))) {
        uc_perror("uc_reg_read_batch", err);
        return 1;
    }

    printf("reg_read_batch = {");

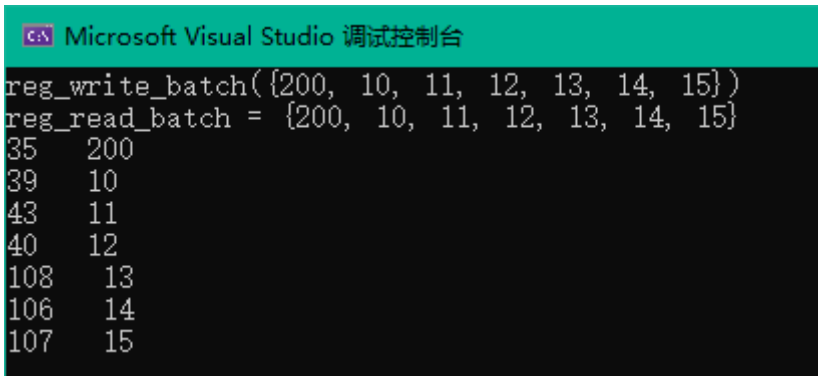
    for (i = 0; i < 7; i++) {
        if (i != 0) printf(", ");
        printf("%" PRIu64, vals[i]);
    }

    printf("}\n");

    uint64_t var[7] = { 0 };
    for (int i = 0; i < 7; i++)
    {
        cout << syscall_abi[i] << "    ";
        printf("%" PRIu64, vals[i]);
        cout << endl;
    }

    return 0;
}
```

输出



uc_reg_read_batch

```
uc_err uc_reg_read_batch(uc_engine *uc, int *regs, void **vals, int count);
```

同时读取多个寄存器的值。

```
@uc: uc_open()返回的句柄
@regid:  存储将被读取的多个寄存器ID的数组
@value:  指向保存多个值的数组的指针
@count: *regs 和 *vals 数组的长度

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_reg_read_batch(uc_engine *uc, int *ids, void **vals, int count)
{
    if (uc->reg_read)
        uc->reg_read(uc, (unsigned int *)ids, vals, count);
    else
        return -1;

    return UC_ERR_OK;
}
```

使用示例同uc_reg_write_batch()。

uc_mem_write

```
uc_err uc_mem_write(uc_engine *uc, uint64_t address, const void *bytes, size_t
size);
```

在内存中写入一段字节码。

```
@uc: uc_open() 返回的句柄
@address: 写入字节的起始地址
@bytes:   指向一个包含要写入内存的数据的指针
@size:   要写入的内存大小。

注意: @bytes 必须足够大以包含 @size 字节。

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_mem_write(uc_engine *uc, uint64_t address, const void *_bytes, size_t
size)
{
    size_t count = 0, len;
    const uint8_t *bytes = _bytes;

    if (uc->mem_redirect) {
```

```
        address = uc->mem_redirect(address);
    }

    if (!check_mem_area(uc, address, size))
        return UC_ERR_WRITE_UNMAPPED;

    // 内存区域可以重叠相邻的内存块
    while(count < size) {
        MemoryRegion *mr = memory_mapping(uc, address);
        if (mr) {
            uint32_t operms = mr->perms;
            if (!(operms & UC_PROT_WRITE)) // 没有写保护
                // 标记为可写
                uc->readonly_mem(mr, false);

            len = (size_t)MIN(size - count, mr->end - address);
            if (uc->write_mem(&uc->as, address, bytes, len) == false)
                break;

            if (!(operms & UC_PROT_WRITE)) // 没有写保护
                // 设置写保护
                uc->readonly_mem(mr, true);

            count += len;
            address += len;
            bytes += len;
        } else  // 此地址尚未被映射
            break;
    }

    if (count == size)
        return UC_ERR_OK;
    else
        return UC_ERR_WRITE_UNMAPPED;
}
```

使用示例:

```
#include <iostream>
#include <string>
#include "unicorn/unicorn.h"
using namespace std;

#define X86_CODE32 "\x41\x4a" // INC ecx; DEC edx
#define ADDRESS 0x1000

int main()
{
    uc_engine* uc;
    uc_err err;

    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }
```

```
    uc_mem_map(uc, ADDRESS, 2 * 1024 * 1024, UC_PROT_ALL);

    if (uc_mem_write(uc, ADDRESS, X86_CODE32, sizeof(X86_CODE32) - 1)) {
        printf("Failed to write emulation code to memory, quit!\n");
        return -1;
    }

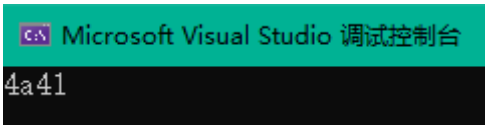
    uint32_t code;

    if(uc_mem_read(uc,ADDRESS,&code, sizeof(code))) {
        printf("Failed to read emulation code to memory, quit!\n");
        return -1;
    }

    cout << hex << code << endl;

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }
    return 0;
}
```

输出



uc_mem_read

```
uc_err uc_mem_read(uc_engine *uc, uint64_t address, void *bytes, size_t size);
```

从内存中读取字节。

```
 @uc: uc_open() 返回的句柄
 @address: 读取字节的起始地址
 @bytes:   指向一个包含要读取内存的数据的指针
 @size:    要读取的内存大小。

 注意: @bytes 必须足够大以包含 @size 字节。

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_mem_read(uc_engine *uc, uint64_t address, void *_bytes, size_t size)
{
    size_t count = 0, len;
    uint8_t *bytes = _bytes;

    if (uc->mem_redirect) {
        address = uc->mem_redirect(address);
```

```
    }

    if (!check_mem_area(uc, address, size))
        return UC_ERR_READ_UNMAPPED;

    // 内存区域可以重叠相邻的内存块
    while(count < size) {
        MemoryRegion *mr = memory_mapping(uc, address);
        if (mr) {
            len = (size_t)MIN(size - count, mr->end - address);
            if (uc->read_mem(&uc->as, address, bytes, len) == false)
                break;
            count += len;
            address += len;
            bytes += len;
        } else  // 此地址尚未被映射
            break;
    }

    if (count == size)
        return UC_ERR_OK;
    else
        return UC_ERR_READ_UNMAPPED;
}
```

使用示例同uc_mem_write()

uc_emu_start

```
uc_err uc_emu_start(uc_engine *uc, uint64_t begin, uint64_t until, uint64_t
timeout, size_t count);
```

在指定的时间内模拟机器码。

```
@uc: uc_open() 返回的句柄
@begin: 开始模拟的地址
@until: 模拟停止的地址 (当到达该地址时)
@timeout: 模拟代码的持续时间(以微秒计)。当这个值为0时，将无时间限制模拟代码，直到模拟完成。
@count: 要模拟的指令数。当这个值为0时，将模拟所有可执行的代码，直到模拟完成

@return 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_emu_start(uc_engine* uc, uint64_t begin, uint64_t until, uint64_t
timeout, size_t count)
{
    // 重制计数器
    uc->emu_counter = 0;
    uc->invalid_error = UC_ERR_OK;
    uc->block_full = false;
    uc->emulation_done = false;
    uc->timed_out = false;
```

```
    switch(uc->arch) {
        default:
            break;
#ifdef UNICORN_HAS_M68K
        case UC_ARCH_M68K:
            uc_reg_write(uc, UC_M68K_REG_PC, &begin);
            break;
#endif
#ifdef UNICORN_HAS_X86
        case UC_ARCH_X86:
            switch(uc->mode) {
                default:
                    break;
                case UC_MODE_16: {
                    uint64_t ip;
                    uint16_t cs;

                    uc_reg_read(uc, UC_X86_REG_CS, &cs);
                    // 抵消后面增加的 IP 和 CS
                    ip = begin - cs*16;
                    uc_reg_write(uc, UC_X86_REG_IP, &ip);
                    break;
                }
                case UC_MODE_32:
                    uc_reg_write(uc, UC_X86_REG_EIP, &begin);
                    break;
                case UC_MODE_64:
                    uc_reg_write(uc, UC_X86_REG_RIP, &begin);
                    break;
            }
            break;
#endif
#ifdef UNICORN_HAS_ARM
        case UC_ARCH_ARM:
            uc_reg_write(uc, UC_ARM_REG_R15, &begin);
            break;
#endif
#ifdef UNICORN_HAS_ARM64
        case UC_ARCH_ARM64:
            uc_reg_write(uc, UC_ARM64_REG_PC, &begin);
            break;
#endif
#ifdef UNICORN_HAS_MIPS
        case UC_ARCH_MIPS:
            // TODO: MIPS32/MIPS64/BIGENDIAN etc
            uc_reg_write(uc, UC_MIPS_REG_PC, &begin);
            break;
#endif
#ifdef UNICORN_HAS_SPARC
        case UC_ARCH_SPARC:
            // TODO: Sparc/Sparc64
            uc_reg_write(uc, UC_SPARC_REG_PC, &begin);
            break;
#endif
    }

    uc->stop_request = false;
```

```
    uc->emu_count = count;
    // 如果不需要计数，则移除计数挂钩hook
    if (count <= 0 && uc->count_hook != 0) {
        uc_hook_del(uc, uc->count_hook);
        uc->count_hook = 0;
    }
    // 设置计数hook记录指令数
    if (count > 0 && uc->count_hook == 0) {
        uc_err err;
        // 对计数指令的回调必须在所有其他操作之前运行，因此必须在hook列表的开头插入hook，而不
是附加hook
        uc->hook_insert = 1;
        err = uc_hook_add(uc, &uc->count_hook, UC_HOOK_CODE, hook_count_cb,
NULL, 1, 0);
        // 恢复到 uc_hook_add()
        uc->hook_insert = 0;
        if (err != UC_ERR_OK) {
            return err;
        }
    }

    uc->addr_end = until;

    if (timeout)
        enable_emu_timer(uc, timeout * 1000);   // microseconds -> nanoseconds

    if (uc->vm_start(uc)) {
        return UC_ERR_RESOURCE;
    }

    // 模拟完成
    uc->emulation_done = true;

    if (timeout) {
        // 等待超时
        qemu_thread_join(&uc->timer);
    }

    if(uc->timed_out)
        return UC_ERR_TIMEOUT;

    return uc->invalid_error;
}
```

使用示例：

```
#include <iostream>
#include <string>
#include "unicorn/unicorn.h"
using namespace std;

#define X86_CODE32 "\x33\xC0" // xor  eax, eax
#define ADDRESS 0x1000

int main()
{
    uc_engine* uc;
```

```
    uc_err err;

    int r_eax = 0x111;

    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return -1;
    }

    uc_mem_map(uc, ADDRESS, 2 * 1024 * 1024, UC_PROT_ALL);

    if (uc_mem_write(uc, ADDRESS, X86_CODE32, sizeof(X86_CODE32) - 1)) {
        printf("Failed to write emulation code to memory, quit!\n");
        return -1;
    }

    uc_reg_write(uc, UC_X86_REG_EAX, &r_eax);
    printf(">>> before EAX = 0x%x\n", r_eax);

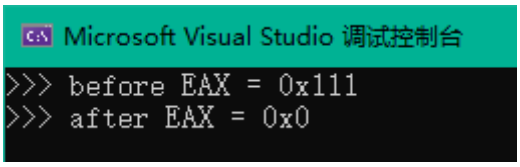
    err = uc_emu_start(uc, ADDRESS, ADDRESS + sizeof(X86_CODE32) - 1, 0, 0);
    if (err) {
        printf("Failed on uc_emu_start() with error returned %u: %s\n",
        err, uc_strerror(err));
    }

    uc_reg_read(uc, UC_X86_REG_EAX, &r_eax);
    printf(">>> after EAX = 0x%x\n", r_eax);

    err = uc_close(uc);
    if (err != UC_ERR_OK) {
        printf("Failed on uc_close() with error returned: %u\n", err);
        return -1;
    }

    return 0;
}
```

输出



uc_emu_stop

```
uc_err uc_emu_stop(uc_engine *uc);
```

停止模拟

通常是从通过 tracing API注册的回调函数中调用。

```
@uc: uc_open() 返回的句柄

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_emu_stop(uc_engine *uc)
{
    if (uc->emulation_done)
        return UC_ERR_OK;

    uc->stop_request = true;

    if (uc->current_cpu) {
        // 退出当前线程
        cpu_exit(uc->current_cpu);
    }

    return UC_ERR_OK;
}
```

使用示例:

```
uc_emu_stop(uc);
```

uc_hook_add

```
uc_err uc_hook_add(uc_engine *uc, uc_hook *hh, int type, void *callback,
        void *user_data, uint64_t begin, uint64_t end, ...);
```

注册hook事件的回调，当hook事件被触发将会进行回调。

```
 @uc: uc_open() 返回的句柄
 @hh: 注册hook得到的句柄. uc_hook_del() 中使用
 @type: hook 类型
 @callback: 当指令被命中时要运行的回调
 @user_data: 用户自定义数据. 将被传递给回调函数的最后一个参数 @user_data
 @begin: 回调生效区域的起始地址(包括)
 @end: 回调生效区域的结束地址(包括)
   注意 1: 只有回调的地址在[@begin, @end]中才会调用回调
   注意 2: 如果 @begin > @end, 每当触发此hook类型时都会调用回调
 @...: 变量参数 (取决于 @type)
   注意: 如果 @type = UC_HOOK_INSN, 这里是指令ID (如: UC_X86_INS_OUT)

 @return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_hook_add(uc_engine *uc, uc_hook *hh, int type, void *callback,
        void *user_data, uint64_t begin, uint64_t end, ...)
{
    int ret = UC_ERR_OK;
```

```
    int i = 0;

    struct hook *hook = calloc(1, sizeof(struct hook));
    if (hook == NULL) {
        return UC_ERR_NOMEM;
    }

    hook->begin = begin;
    hook->end = end;
    hook->type = type;
    hook->callback = callback;
    hook->user_data = user_data;
    hook->refs = 0;
    *hh = (uc_hook)hook;

    // UC_HOOK_INSN 有一个额外参数：指令ID
    if (type & UC_HOOK_INSN) {
        va_list valist;

        va_start(valist, end);
        hook->insn = va_arg(valist, int);
        va_end(valist);

        if (uc->insn_hook_validate) {
            if (! uc->insn_hook_validate(hook->insn)) {
                free(hook);
                return UC_ERR_HOOK;
            }
        }

        if (uc->hook_insert) {
            if (list_insert(&uc->hook[UC_HOOK_INSN_IDX], hook) == NULL) {
                free(hook);
                return UC_ERR_NOMEM;
            }
        } else {
            if (list_append(&uc->hook[UC_HOOK_INSN_IDX], hook) == NULL) {
                free(hook);
                return UC_ERR_NOMEM;
            }
        }

        hook->refs++;
        return UC_ERR_OK;
    }

    while ((type >> i) > 0) {
        if ((type >> i) & 1) {
            if (i < UC_HOOK_MAX) {
                if (uc->hook_insert) {
                    if (list_insert(&uc->hook[i], hook) == NULL) {
                        if (hook->refs == 0) {
                            free(hook);
                        }
                        return UC_ERR_NOMEM;
                    }
                } else {
                    if (list_append(&uc->hook[i], hook) == NULL) {
```

```
                        if (hook->refs == 0) {
                            free(hook);
                        }
                        return UC_ERR_NOMEM;
                    }
                }
                hook->refs++;
            }
        }
        i++;
    }

    if (hook->refs == 0) {
        free(hook);
    }

    return ret;
}
```

使用示例：

```
#include <iostream>
#include <string>
#include "unicorn/unicorn.h"
using namespace std;

int syscall_abi[] = {
    UC_X86_REG_RAX, UC_X86_REG_RDI, UC_X86_REG_RSI, UC_X86_REG_RDX,
    UC_X86_REG_R10, UC_X86_REG_R8, UC_X86_REG_R9
};

uint64_t vals[7] = { 200, 10, 11, 12, 13, 14, 15 };

void* ptrs[7];

void uc_perror(const char* func, uc_err err)
{
    fprintf(stderr, "Error in %s(): %s\n", func, uc_strerror(err));
}

#define BASE 0x10000

// mov rax, 100; mov rdi, 1; mov rsi, 2; mov rdx, 3; mov r10, 4; mov r8, 5; mov
r9, 6; syscall
#define CODE
"\x48\xc7\xc0\x64\x00\x00\x00\x48\xc7\xc7\x01\x00\x00\x00\x48\xc7\xc6\x02\x00\x0
0\x00\x48\xc7\xc2\x03\x00\x00\x00\x49\xc7\xc2\x04\x00\x00\x00\x49\xc7\xc0\x05\x0
0\x00\x00\x49\xc7\xc1\x06\x00\x00\x00\x0f\x05"

void hook_syscall(uc_engine* uc, void* user_data)
{
    int i;

    uc_reg_read_batch(uc, syscall_abi, ptrs, 7);

    printf("syscall: {");
```

```
    for (i = 0; i < 7; i++) {
        if (i != 0) printf(", ");
        printf("%" PRIu64, vals[i]);
    }

    printf("}\n");
}

void hook_code(uc_engine* uc, uint64_t addr, uint32_t size, void* user_data)
{
    printf("HOOK_CODE: 0x%" PRIx64 ", 0x%x\n", addr, size);
}

int main()
{
    int i;
    uc_hook sys_hook;
    uc_err err;
    uc_engine* uc;

    for (i = 0; i < 7; i++) {
        ptrs[i] = &vals[i];
    }

    if ((err = uc_open(UC_ARCH_X86, UC_MODE_64, &uc))) {
        uc_perror("uc_open", err);
        return 1;
    }

    printf("reg_write_batch({200, 10, 11, 12, 13, 14, 15})\n");
    if ((err = uc_reg_write_batch(uc, syscall_abi, ptrs, 7))) {
        uc_perror("uc_reg_write_batch", err);
        return 1;
    }

    memset(vals, 0, sizeof(vals));
    if ((err = uc_reg_read_batch(uc, syscall_abi, ptrs, 7))) {
        uc_perror("uc_reg_read_batch", err);
        return 1;
    }

    printf("reg_read_batch = {");

    for (i = 0; i < 7; i++) {
        if (i != 0) printf(", ");
        printf("%" PRIu64, vals[i]);
    }

    printf("}\n");

    // syscall
    printf("\n");
    printf("running syscall shellcode\n");

    if ((err = uc_hook_add(uc, &sys_hook, UC_HOOK_CODE, hook_syscall, NULL, 1,
0))) {
        uc_perror("uc_hook_add", err);
        return 1;
```

```
    }

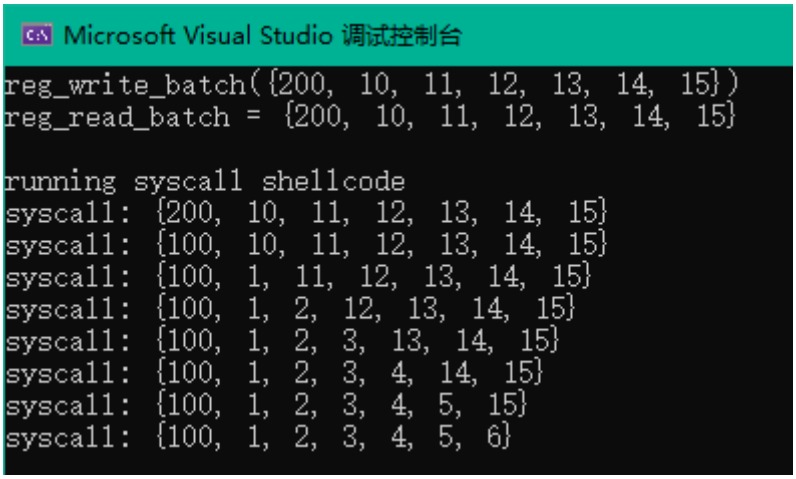
    if ((err = uc_mem_map(uc, BASE, 0x1000, UC_PROT_ALL))) {
        uc_perror("uc_mem_map", err);
        return 1;
    }

    if ((err = uc_mem_write(uc, BASE, CODE, sizeof(CODE) - 1))) {
        uc_perror("uc_mem_write", err);
        return 1;
    }

    if ((err = uc_emu_start(uc, BASE, BASE + sizeof(CODE) - 1, 0, 0))) {
        uc_perror("uc_emu_start", err);
        return 1;
    }

    return 0;
}
```

输出



对每条指令都进行hook

uc_hook_del

```
uc_err uc_hook_del(uc_engine *uc, uc_hook hh);
```

删除一个已注册的hook事件

```
@uc: uc_open() 返回的句柄
@hh: uc_hook_add() 返回的句柄

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_hook_del(uc_engine *uc, uc_hook hh)
{
    int i;
```

```
    struct hook *hook = (struct hook *)hh;

    for (i = 0; i < UC_HOOK_MAX; i++) {
        if (list_remove(&uc->hook[i], (void *)hook)) {
            if (--hook->refs == 0) {
                free(hook);
                break;
            }
        }
    }
    return UC_ERR_OK;
}
```

使用示例：

```
if ((err = uc_hook_add(uc, &sys_hook, UC_HOOK_CODE, hook_syscall, NULL, 1, 0)))
{
    uc_perror("uc_hook_add", err);
    return 1;
}

if ((err = uc_hook_del(uc, &sys_hook))) {
    uc_perror("uc_hook_del", err);
    return 1;
}
```

uc_mem_map

```
uc_err uc_mem_map(uc_engine *uc, uint64_t address, size_t size, uint32_t perms);
```

为模拟映射一块内存。

```
@uc: uc_open() 返回的句柄
@address: 要映射到的新内存区域的起始地址。这个地址必须与4KB对齐，否则将返回UC_ERR_ARG错误。
@size: 要映射到的新内存区域的大小。这个大小必须是4KB的倍数，否则将返回UC_ERR_ARG错误。
@perms: 新映射区域的权限。参数必须是UC_PROT_READ | UC_PROT_WRITE | UC_PROT_EXEC或这些的
组合，否则返回UC_ERR_ARG错误。

@return 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_mem_map(uc_engine *uc, uint64_t address, size_t size, uint32_t perms)
{
    uc_err res;

    if (uc->mem_redirect) {
        address = uc->mem_redirect(address);
    }

    res = mem_map_check(uc, address, size, perms);    //内存安全检查
    if (res)
        return res;
```

```
    return mem_map(uc, address, size, perms, uc->memory_map(uc, address, size,
perms));
}
```

使用示例同 uc_hook_add()

uc_mem_map_ptr

```
uc_err uc_mem_map_ptr(uc_engine *uc, uint64_t address, size_t size, uint32_t
perms, void *ptr);
```

在模拟中映射现有的主机内存。

```
@uc: uc_open() 返回的句柄
@address: 要映射到的新内存区域的起始地址。这个地址必须与4KB对齐，否则将返回UC_ERR_ARG错误。
@size: 要映射到的新内存区域的大小。这个大小必须是4KB的倍数，否则将返回UC_ERR_ARG错误。
@perms: 新映射区域的权限。参数必须是UC_PROT_READ | UC_PROT_WRITE | UC_PROT_EXEC或这些的
组合，否则返回UC_ERR_ARG错误。
@ptr: 指向支持新映射内存的主机内存的指针。映射的主机内存的大小应该与size的大小相同或更大，并且
至少使用PROT_READ | PROT_WRITE进行映射，否则不定义映射。

@return 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_mem_map_ptr(uc_engine *uc, uint64_t address, size_t size, uint32_t
perms, void *ptr)
{
    uc_err res;

    if (ptr == NULL)
        return UC_ERR_ARG;

    if (uc->mem_redirect) {
        address = uc->mem_redirect(address);
    }

    res = mem_map_check(uc, address, size, perms);    //内存安全检查
    if (res)
        return res;

    return mem_map(uc, address, size, UC_PROT_ALL, uc->memory_map_ptr(uc,
address, size, perms, ptr));
}
```

使用示例同 uc_mem_map()

uc_mem_unmap

```
uc_err uc_mem_unmap(uc_engine *uc, uint64_t address, size_t size);
```

取消对模拟内存区域的映射

```
@uc: uc_open() 返回的句柄
@address: 要映射到的新内存区域的起始地址。这个地址必须与4KB对齐，否则将返回UC_ERR_ARG错误。
@size: 要映射到的新内存区域的大小。这个大小必须是4KB的倍数，否则将返回UC_ERR_ARG错误。

@return 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_mem_unmap(struct uc_struct *uc, uint64_t address, size_t size)
{
    MemoryRegion *mr;
    uint64_t addr;
    size_t count, len;

    if (size == 0)
        // 没有要取消映射的区域
        return UC_ERR_OK;

    // 地址必须对齐到 uc->target_page_size
    if ((address & uc->target_page_align) != 0)
        return UC_ERR_ARG;

    // 大小必须是 uc->target_page_size 的倍数
    if ((size & uc->target_page_align) != 0)
        return UC_ERR_ARG;

    if (uc->mem_redirect) {
        address = uc->mem_redirect(address);
    }

    // 检查用户请求的整个块是否被映射
    if (!check_mem_area(uc, address, size))
        return UC_ERR_NOMEM;

    // 如果这个区域跨越了相邻的区域，可能需要分割区域
    addr = address;
    count = 0;
    while(count < size) {
        mr = memory_mapping(uc, addr);
        len = (size_t)MIN(size - count, mr->end - addr);
        if (!split_region(uc, mr, addr, len, true))
            return UC_ERR_NOMEM;

        // 取消映射
        mr = memory_mapping(uc, addr);
        if (mr != NULL)
           uc->memory_unmap(uc, mr);
        count += len;
        addr += len;
```

```
    }

    return UC_ERR_OK;
}
```

使用示例：

```
if ((err = uc_mem_map(uc, BASE, 0x1000, UC_PROT_ALL))) {
    uc_perror("uc_mem_map", err);
    return 1;
}

if ((err = uc_mem_unmap(uc, BASE, 0x1000))) {
    uc_perror("uc_mem_unmap", err);
    return 1;
}
```

uc_mem_protect

```
uc_err uc_mem_protect(uc_engine *uc, uint64_t address, size_t size, uint32_t
perms);
```

设置模拟内存的权限

```
@uc: uc_open() 返回的句柄
@address: 要映射到的新内存区域的起始地址。这个地址必须与4KB对齐，否则将返回UC_ERR_ARG错误。
@size: 要映射到的新内存区域的大小。这个大小必须是4KB的倍数，否则将返回UC_ERR_ARG错误。
@perms: 映射区域的新权限。参数必须是UC_PROT_READ | UC_PROT_WRITE | UC_PROT_EXEC或这些的
组合，否则返回UC_ERR_ARG错误。

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_mem_protect(struct uc_struct *uc, uint64_t address, size_t size,
uint32_t perms)
{
    MemoryRegion *mr;
    uint64_t addr = address;
    size_t count, len;
    bool remove_exec = false;

    if (size == 0)
        // trivial case, no change
        return UC_ERR_OK;

    // address must be aligned to uc->target_page_size
    if ((address & uc->target_page_align) != 0)
        return UC_ERR_ARG;

    // size must be multiple of uc->target_page_size
    if ((size & uc->target_page_align) != 0)
        return UC_ERR_ARG;
```

```
    // check for only valid permissions
    if ((perms & ~UC_PROT_ALL) != 0)
        return UC_ERR_ARG;

    if (uc->mem_redirect) {
        address = uc->mem_redirect(address);
    }

    // check that user's entire requested block is mapped
    if (!check_mem_area(uc, address, size))
        return UC_ERR_NOMEM;

    // Now we know entire region is mapped, so change permissions
    // We may need to split regions if this area spans adjacent regions
    addr = address;
    count = 0;
    while(count < size) {
        mr = memory_mapping(uc, addr);
        len = (size_t)MIN(size - count, mr->end - addr);
        if (!split_region(uc, mr, addr, len, false))
            return UC_ERR_NOMEM;

        mr = memory_mapping(uc, addr);
        // will this remove EXEC permission?
        if (((mr->perms & UC_PROT_EXEC) != 0) && ((perms & UC_PROT_EXEC) == 0))
            remove_exec = true;
        mr->perms = perms;
        uc->readonly_mem(mr, (perms & UC_PROT_WRITE) == 0);

        count += len;
        addr += len;
    }

    // if EXEC permission is removed, then quit TB and continue at the same
place
    if (remove_exec) {
        uc->quit_request = true;
        uc_emu_stop(uc);
    }

    return UC_ERR_OK;
}
```

使用示例：

```
if ((err = uc_mem_protect(uc, BASE, 0x1000, UC_PROT_ALL))) {  //可读可写可执行
    uc_perror("uc_mem_protect", err);
    return 1;
}
```

uc_mem_regions

```
uc_err uc_mem_regions(uc_engine *uc, uc_mem_region **regions, uint32_t *count);
```

检索由 uc_mem_map() 和 uc_mem_map_ptr() 映射的内存的信息。

这个API为@regions分配内存，用户之后必须通过free()释放这些内存来避免内存泄漏。

```
@uc: uc_open() 返回的句柄
@regions: 指向 uc_mem_region 结构体的数组的指针. 由Unicorn申请，必须通过uc_free()释放这些内存
@count: 指向@regions中包含的uc_mem_region结构体的数量的指针

@return 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

源码分析

▶ Code

```
uint32_t uc_mem_regions(uc_engine *uc, uc_mem_region **regions, uint32_t *count)
{
    uint32_t i;
    uc_mem_region *r = NULL;

    *count = uc->mapped_block_count;

    if (*count) {
        r = g_malloc0(*count * sizeof(uc_mem_region));
        if (r == NULL) {
            // 内存不足
            return UC_ERR_NOMEM;
        }
    }

    for (i = 0; i < *count; i++) {
        r[i].begin = uc->mapped_blocks[i]->addr;
        r[i].end = uc->mapped_blocks[i]->end - 1;
        r[i].perms = uc->mapped_blocks[i]->perms;
    }

    *regions = r;

    return UC_ERR_OK;
}
```

使用示例:

```
#include <iostream>
#include <string>
#include "unicorn/unicorn.h"
using namespace std;

int main()
{
    uc_err err;
```

```
    uc_engine* uc;

    if ((err = uc_open(UC_ARCH_X86, UC_MODE_64, &uc))) {
        uc_perror("uc_open", err);
        return 1;
    }

    if ((err = uc_mem_map(uc, BASE, 0x1000, UC_PROT_ALL))) {
        uc_perror("uc_mem_map", err);
        return 1;
    }

    uc_mem_region *region;
    uint32_t count;

    if ((err = uc_mem_regions(uc, &region, &count))) {
        uc_perror("uc_mem_regions", err);
        return 1;
    }

    cout << "起始地址：  0x" << hex << region->begin << "  结束地址：  0x" << hex <<
region->end << "  内存权限：   " <<region->perms << "  已申请内存块数：  " << count <<
endl;

    if ((err = uc_free(region))) {    ////注意释放内存
        uc_perror("uc_free", err);
        return 1;
    }

    return 0;
}
```

输出



uc_free

```
uc_err uc_free(void *mem);
```

释放由 uc_mem_regions() 申请的内存

```
@mem: 由 uc_mem_regions (返回 *regions)申请的内存

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_free(void *mem)
{
    g_free(mem);
    return UC_ERR_OK;
}

void g_free(gpointer ptr)
{
   free(ptr);
}
```

使用示例同 uc_mem_regions()

uc_context_alloc

```
uc_err uc_context_alloc(uc_engine *uc, uc_context **context);
```

分配一个可以与uc_context_{save,restore}一起使用的区域来执行CPU上下文的快速保存/回滚，包括寄存器和内部元数据。上下文不能在具有不同架构或模式的引擎实例之间共享。

```
@uc: uc_open() 返回的句柄
@context: 指向uc_engine*的指针。当这个函数成功返回时，将使用指向新上下文的指针更新它。之后必须使用uc_context_free()释放这些分配的内存。

@return 成功则返回UC_ERR_OK ，否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_context_alloc(uc_engine *uc, uc_context **context)
{
    struct uc_context **_context = context;
    size_t size = uc->cpu_context_size;

    *_context = g_malloc(size);
    if (*_context) {
        (*_context)->jmp_env_size = sizeof(*uc->cpu->jmp_env);
        (*_context)->context_size = size - sizeof(uc_context) - (*_context)-
>jmp_env_size;
        return UC_ERR_OK;
    } else {
        return UC_ERR_NOMEM;
    }
}
```

使用示例

```
#include <iostream>
#include <string>
#include "unicorn/unicorn.h"
using namespace std;

#define ADDRESS 0x1000
```

```c
#define X86_CODE32_INC "\x40"   // INC eax

int main()
{
    uc_engine* uc;
    uc_context* context;
    uc_err err;

    int r_eax = 0x1;    // EAX 寄存器

    printf("=================================\n");
    printf("Save/restore CPU context in opaque blob\n");

    err = uc_open(UC_ARCH_X86, UC_MODE_32, &uc);
    if (err) {
        printf("Failed on uc_open() with error returned: %u\n", err);
        return 0;
    }

    uc_mem_map(uc, ADDRESS, 8 * 1024, UC_PROT_ALL);

    if (uc_mem_write(uc, ADDRESS, X86_CODE32_INC, sizeof(X86_CODE32_INC) - 1)) {
        printf("Failed to write emulation code to memory, quit!\n");
        return 0;
    }

    // 初始化寄存器
    uc_reg_write(uc, UC_X86_REG_EAX, &r_eax);

    printf(">>> Running emulation for the first time\n");

    err = uc_emu_start(uc, ADDRESS, ADDRESS + sizeof(X86_CODE32_INC) - 1, 0, 0);
    if (err) {
        printf("Failed on uc_emu_start() with error returned %u: %s\n",
            err, uc_strerror(err));
    }

    printf(">>> Emulation done. Below is the CPU context\n");

    uc_reg_read(uc, UC_X86_REG_EAX, &r_eax);
    printf(">>> EAX = 0x%x\n", r_eax);

    // 申请并保存 CPU 上下文
    printf(">>> Saving CPU context\n");

    err = uc_context_alloc(uc, &context);
    if (err) {
        printf("Failed on uc_context_alloc() with error returned: %u\n", err);
        return 0;
    }

    err = uc_context_save(uc, context);
    if (err) {
        printf("Failed on uc_context_save() with error returned: %u\n", err);
        return 0;
    }

    printf(">>> Running emulation for the second time\n");
```

```
    err = uc_emu_start(uc, ADDRESS, ADDRESS + sizeof(X86_CODE32_INC) - 1, 0, 0);
    if (err) {
        printf("Failed on uc_emu_start() with error returned %u: %s\n",
            err, uc_strerror(err));
    }

    printf(">>> Emulation done. Below is the CPU context\n");

    uc_reg_read(uc, UC_X86_REG_EAX, &r_eax);
    printf(">>> EAX = 0x%x\n", r_eax);

    // 恢复 CPU 上下文
    err = uc_context_restore(uc, context);
    if (err) {
        printf("Failed on uc_context_restore() with error returned: %u\n", err);
        return 0;
    }

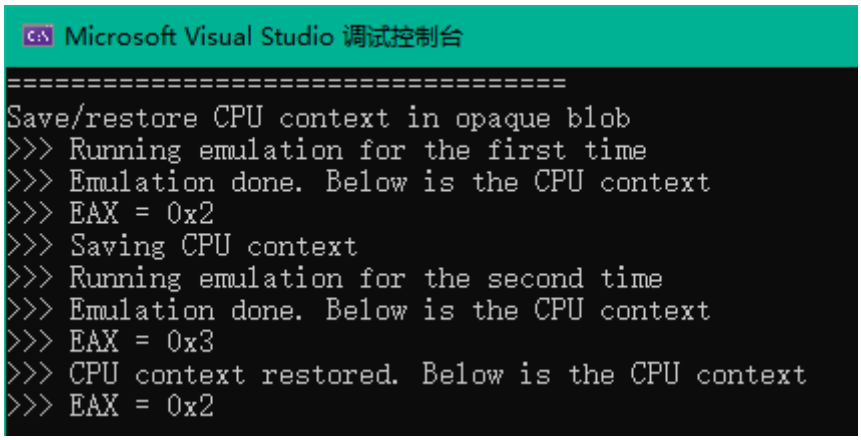
    printf(">>> CPU context restored. Below is the CPU context\n");

    uc_reg_read(uc, UC_X86_REG_EAX, &r_eax);
    printf(">>> EAX = 0x%x\n", r_eax);

    // 释放 CPU 上下文
    err = uc_context_free(context);
    if (err) {
        printf("Failed on uc_free() with error returned: %u\n", err);
        return;
    }

    uc_close(uc);
}
```

输出



uc_context_save

```
uc_err uc_context_save(uc_engine *uc, uc_context *context);
```

保存当前CPU上下文

```
@uc: uc_open() 返回的句柄
@context: uc_context_alloc() 返回的句柄

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_context_save(uc_engine *uc, uc_context *context)
{
    struct uc_context *_context = context;
    memcpy(_context->data, uc->cpu->env_ptr, _context->size);
    return UC_ERR_OK;
}
```

使用示例同 uc_context_alloc()

uc_context_restore

```
uc_err uc_context_restore(uc_engine *uc, uc_context *context);
```

恢复已保存的CPU上下文

```
@uc: uc_open() 返回的句柄
@context: uc_context_alloc() 返回并且已使用 uc_context_save 保存的句柄

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

▶ 源码实现

```
uc_err uc_context_restore(uc_engine *uc, uc_context *context)
{
    struct uc_context *_context = context;
    memcpy(uc->cpu->env_ptr, _context->data, _context->size);
    return UC_ERR_OK;
}
```

使用示例同 uc_context_alloc()

uc_context_size

```
size_t uc_context_size(uc_engine *uc);
```

返回存储cpu上下文所需的大小。可以用来分配一个缓冲区来包含cpu上下文，并直接调用uc_context_save。

```
@uc: uc_open() 返回的句柄

@return 存储cpu上下文所需的大小，类型为 size_t.
```

▶ 源码实现

```
size_t uc_context_size(uc_engine *uc)
{
    return sizeof(uc_context) + uc->cpu_context_size + sizeof(*uc->cpu-
>jmp_env);
}
```

使用示例同 uc_context_alloc()

uc_context_free

```
uc_err uc_context_free(uc_context *context);
```

释放由 uc_context_alloc() 申请的内存

```
@context: 由uc_context_alloc创建的uc_context

@return 成功则返回UC_ERR_OK , 否则返回 uc_err 枚举的其他错误类型
```

使用示例同 uc_context_alloc()